Step 2 — Add a Long-Term Store, and a Node That Reads It — Solution

> Back to the step · Hint 1 · Hint 2

Story: this is the actual fix for Step 1's amnesia — a LangGraph Store keyed by `user_id` instead of `thread_id`, so facts about a person persist across every conversation they ever have, plus a `recall_node` that reads from it *before* the chat node runs, so a brand-new conversation still recognizes the returning user. **If not:** every new thread would meet a complete stranger, no matter how many times that person had talked to it before.

`src/memory/store.py` (new)

Story: these are thin helper functions wrapped around LangGraph's Store API. The namespace itself is keyed by `user_id`, so saved facts belong to the *person*, not to any one conversation thread. Saving a fact under the same key **updates** the existing entry rather than creating a duplicate. **If not:** facts keyed by thread would vanish with each new conversation, exactly the same problem the checkpointer alone had.

```
# src/memory/store.py
# The long-term memory: facts about a user, independent of any conversation.
# Keyed by user_id (not thread_id). Same key on save = update, not duplicate.
from langgraph.store.memory import InMemoryStore


def build_store():
    return InMemoryStore()


def namespace_for(user_id):
    return ("memories", user_id)


def recall_facts(store, user_id):
    facts = []
    for item in store.search(namespace_for(user_id)):
        facts.append(item.value["fact"])
    return facts
```

```
def save_fact(store, user_id, key, fact):
    store.put(namespace_for(user_id), key, {"fact": fact})
```

`src/nodes.py` — add `recall_node`

Story: `recall_node` runs before the chat node, every single turn. It reads the user's saved facts from the Store and puts them into `state["recalled"]`, which `chat_node` then folds directly into its system prompt. The Store actually reaches inside the node because the node function declares a `store` parameter, and the graph itself was compiled with `store=...` passed in. **If not:** the recalled facts would never actually make it into the prompt the model sees, no matter how well they were saved.

```
# src/nodes.py  (add this; chat_node from Step 1 is unchanged)

from src.memory.store import recall_facts

def recall_node(state, *, store):
    facts = recall_facts(store, state["user_id"])
    return {"recalled": "; ".join(facts)}
```

`src/graph.py` — recall before chat

```
# src/graph.py  (Step 2 version)
from langgraph.graph import END, START, StateGraph

from src.nodes import make_chat_node, recall_node
from src.state import ChatState


def build_graph(model, store, checkpointer):
    graph = StateGraph(ChatState)
    graph.add_node("recall", recall_node)
    graph.add_node("chat", make_chat_node(model))
    graph.add_edge(START, "recall")
    graph.add_edge("recall", "chat")
    graph.add_edge("chat", END)
    return graph.compile(checkpointer=checkpointer, store=store)
```

In `main.py`, create the store with `build_store()` and carry a real `user_id`. Seed a fact by hand to see recall work before Step 3 writes them automatically:

```
from src.memory.store import build_store, save_fact
# ...
store = build_store()
save_fact(store, "sam", "preferred_name", "Name is Sam")
# then build the graph with this store; a brand-new thread_id still recalls Sam
```

The test

```
# tests/unit/test_store.py  (Step 2 cases; Step 4 adds the forget case)
# The Store helpers, no model, no graph. Run from the repo root:
#   python -m tests.unit.test_store
from src.memory.store import build_store, recall_facts, save_fact


def test_save_then_recall():
    store = build_store()
    save_fact(store, "sam", "preferred_name", "Name is Sam")
    result = "OK"
    if recall_facts(store, "sam") != ["Name is Sam"]:
        result = "WRONG"
    print(f"case 1 (save then recall)      -> {result}")


def test_recall_is_per_user():
    store = build_store()
    save_fact(store, "sam", "preferred_name", "Name is Sam")
    result = "OK"
    if recall_facts(store, "amy") != []:
        result = "WRONG"
    print(f"case 2 (another user sees none) -> {result}")


def test_same_key_updates():
    store = build_store()
    save_fact(store, "sam", "preferred_name", "Name is Sam")
    save_fact(store, "sam", "preferred_name", "Name is Sammy")
    result = "OK"
    if recall_facts(store, "sam") != ["Name is Sammy"]:
        result = "WRONG"
    print(f"case 3 (same key updates)       -> {result}")


if __name__ == "__main__":
    test_save_then_recall()
```

```
    test_recall_is_per_user()
    test_same_key_updates()
```

Expected output:

```
case 1 (save then recall)      -> OK
case 2 (another user sees none) -> OK
case 3 (same key updates)       -> OK
```

The key insight case 3 proves: because facts are keyed (not just appended), a later value for `preferred_name` **replaces** the old one — the dedup you need before Step 3 starts writing automatically. And because the namespace is `user_id`, a brand-new `thread_id` reading through `recall_node` still finds the user's facts — the amnesia from Step 1 is gone.